Project Planning & Management

Planning, Scheduling, Risk Assessment, and KPIs

Podcast Streaming Platform

Graduation Project - Faculty of Computers and Information
Backend: ASP.NET Core Web API | Frontend: React / Next.js
Methodology: Agile Scrum | Duration: 16 Weeks

Prepared by: Project Development Team

Academic Year 2025 - 2026

Table of Contents

1. Project Proposal
    1.1 Executive Summary
    1.2 Project Objectives
    1.3 Scope Statement
2. Project Plan
    2.1 Methodology & Timeline
    2.2 Gantt Chart
    2.3 Milestones & Deliverables
3. Task Assignment & Roles
    3.1 Team Structure & WBS
    3.2 Resource Allocation
4. Risk Assessment & Mitigation
    4.1 Risk Register
    4.2 Risk Heatmap
    4.3 Mitigation Strategies
5. Key Performance Indicators
    5.1 Project Management KPIs
    5.2 Technical KPIs

1. Project Proposal

1.1 Executive Summary

The Podcast Streaming Platform is a web-based application that enables users to discover, stream, and organize podcast content extracted from YouTube videos. Content creators can submit YouTube URLs, automatically extract audio streams using yt-dlp, apply precise trimming controls through an integrated FFmpeg-based editor, and publish processed episodes to the platform catalog. The system targets three user roles: Listeners (browse/play/organize), Creators (extract/trim/publish), and Administrators (manage/monitor/analytics).

The project follows Agile Scrum methodology over a 16-week timeline with 8 two-week sprints. The tech stack includes ASP.NET Core 8 Web API (backend), React/Next.js 14 (frontend), PostgreSQL 16 (database), Redis (caching), and yt-dlp/FFmpeg (audio processing). The platform is developed by a 4-member team and delivered as a fully functional web application with comprehensive documentation.

1.2 Project Objectives

ID	Objective
O1	Build a responsive web app for podcast discovery, streaming, and content management
O2	Implement YouTube-to-audio conversion with timestamp-based trimming (yt-dlp + FFmpeg)
O3	Provide content creators with metadata management, categorization, and engagement analytics
O4	Ensure high performance (2s page load, 1s search), security (JWT, OWASP), and scalability
O5	Deliver complete, tested, and documented system using Agile Scrum within 16 weeks

Table 1.1: Project objectives with measurable targets.

1.3 Scope Statement

Category	Description
In Scope	User registration/authentication; YouTube audio extraction and trimming; web-based audio player; search and filtering; playlists and favorites; admin dashboard with analytics; responsive design (desktop + mobile)
Out of Scope	Native mobile apps (iOS/Android); live streaming; content monetization; third-party RSS feed import; social media integrations; AI-based recommendations

Table 1.2: Project scope boundaries.

2. Project Plan

2.1 Methodology & Timeline

The project adopts Agile Scrum with 8 two-week sprints over 16 weeks. Each sprint includes planning, development, review, and retrospective ceremonies. The team holds daily 15-minute stand-ups and uses a shared Kanban board for task tracking.

Aspect	Approach	Details
Development	Agile Scrum (8 sprints x 2 weeks)	Iterative delivery with sprint reviews
Communication	Daily stand-ups, bi-weekly demos	Slack + Jira for task management
Version Control	Git with GitHub	Feature branches, PR reviews, CI/CD
Documentation	Concurrent with development	Markdown + auto-generated API docs (Swagger)

Table 2.1: Project management approach.

2.2 Gantt Chart

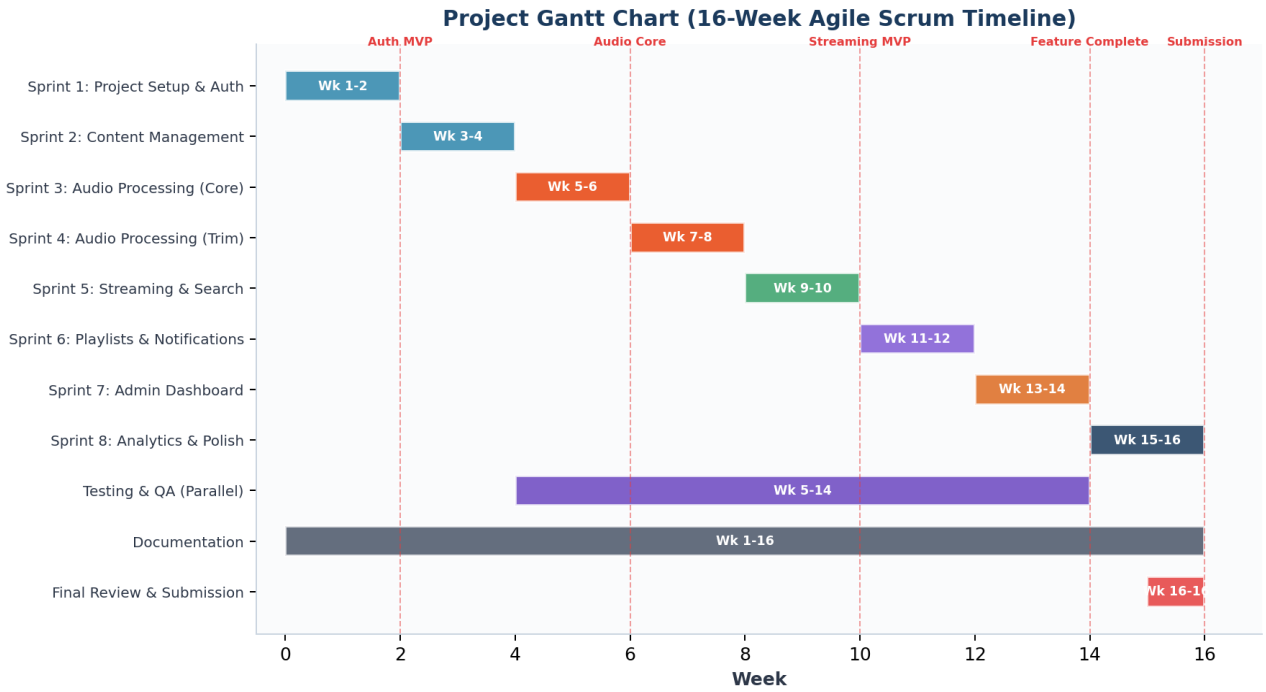

Figure 2.1: Project Gantt chart showing 16-week sprint timeline with milestones.

2.3 Milestones & Deliverables

Milestone	Deliverable	Description
M1 (Week 2)	Auth MVP	User registration, login, JWT tokens functional
M2 (Week 6)	Audio Processing Core	YouTube URL extraction + audio normalization working
M3 (Week 10)	Streaming MVP	Full browse-search-play-listen flow operational
M4 (Week 14)	Feature Complete	All Must Have + Should Have requirements implemented
M5 (Week 16)	Final Submission	Tested, documented, and deployed system ready for review

Table 2.2: Project milestones and key deliverables.

3. Task Assignment & Roles

3.1 Team Structure & WBS

The 4-member team is organized by primary responsibility with cross-functional overlap. Each member has a defined role but contributes across multiple modules as needed.

Member	Role	Key Responsibilities
Member 1	Frontend Developer	React/Next.js UI, Audio Player, Responsive Design, Component Library
Member 2	Backend Developer	ASP.NET Core API, Auth Service, Podcast Service, Database Access Layer
Member 3	Backend + Audio	Audio Processing Pipeline (yt-dlp/FFmpeg), Background Jobs, Redis Queue
Member 4	Full-Stack / QA	Admin Dashboard, Analytics, Search Service, Testing, Documentation

Table 3.1: Team structure with roles and responsibilities.

Work Breakdown Structure (WBS) - Key Work Packages:

WBS ID	Work Package	Scope	Duration	Assigned To
WBS 1.1	Authentication & User Management	Login, Register, JWT, Roles, Profile	2 weeks	Backend Dev
WBS 1.2	Podcast Content Management	CRUD, Categories, Metadata, Moderation	3 weeks	Backend Dev
WBS 1.3	Audio Processing Pipeline	yt-dlp extraction, FFmpeg trim, Job Queue	4 weeks	Backend + Audio
WBS 1.4	Streaming & Playback	Web Audio Player, Progressive Streaming	2 weeks	Frontend Dev
WBS 1.5	Search & Discovery	Full-text Search, Filters, Recommendations	2 weeks	Full-Stack
WBS 1.6	Playlists & Favorites	CRUD, Reorder, Share functionality	1.5 weeks	Frontend Dev
WBS 1.7	Admin Dashboard	User Mgmt, Analytics, Content Moderation	2 weeks	Full-Stack
WBS 1.8	UI/UX Design & Implementation	Wireframes, Design System, Responsive Layout	Ongoing	Frontend Dev
WBS 1.9	Testing & Quality Assurance	Unit, Integration, E2E, Performance Tests	Ongoing	QA Engineer
WBS 1.10	Documentation & Deployment	API Docs, User Guide, CI/CD, Docker	Ongoing	All Members

Table 3.2: Work Breakdown Structure with 10 major work packages.

3.2 Resource Allocation

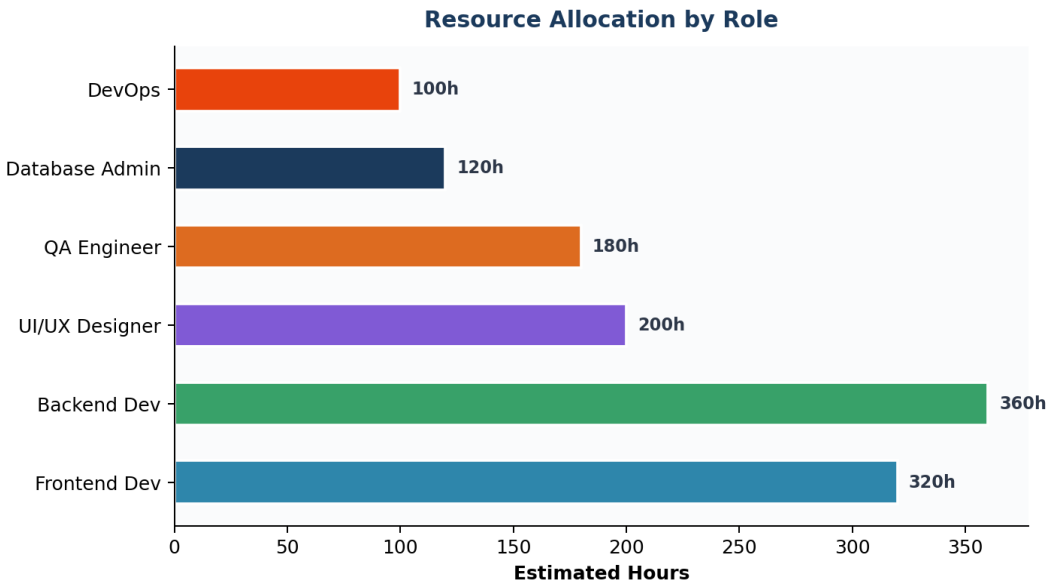

Figure 3.1: Estimated effort allocation by team role (total ~1,280 hours).

4. Risk Assessment & Mitigation

4.1 Risk Register

12 risks identified across technical, operational, and project management categories. Each risk is assessed on Probability (1-5) and Impact (1-5) scales.

ID	Risk	Prob.	Impact	Score	Priority
R01	Audio processing failure	Medium	Critical	3x5=15	High
R02	YouTube API changes/limits	Low	Major	2x4=8	Medium
R03	Security vulnerability	Very Low	Critical	1x5=5	Medium
R04	Scope creep	Medium	Moderate	3x3=9	Medium
R05	Team member unavailability	Low	Moderate	2x3=6	Medium
R06	Performance bottlenecks	Medium	Major	3x4=12	High
R07	Database data loss	Very Low	Critical	1x5=5	Medium
R08	Timeline overrun	High	Major	4x4=16	High
R09	Requirement ambiguity	Medium	Minor	3x2=6	Low
R10	Third-party service outage	Low	Moderate	2x3=6	Low
R11	Insufficient testing coverage	Medium	Major	3x4=12	High
R12	Technology learning curve	Low	Minor	2x2=4	Low

Table 4.1: Risk register with probability-impact scoring.

4.2 Risk Heatmap

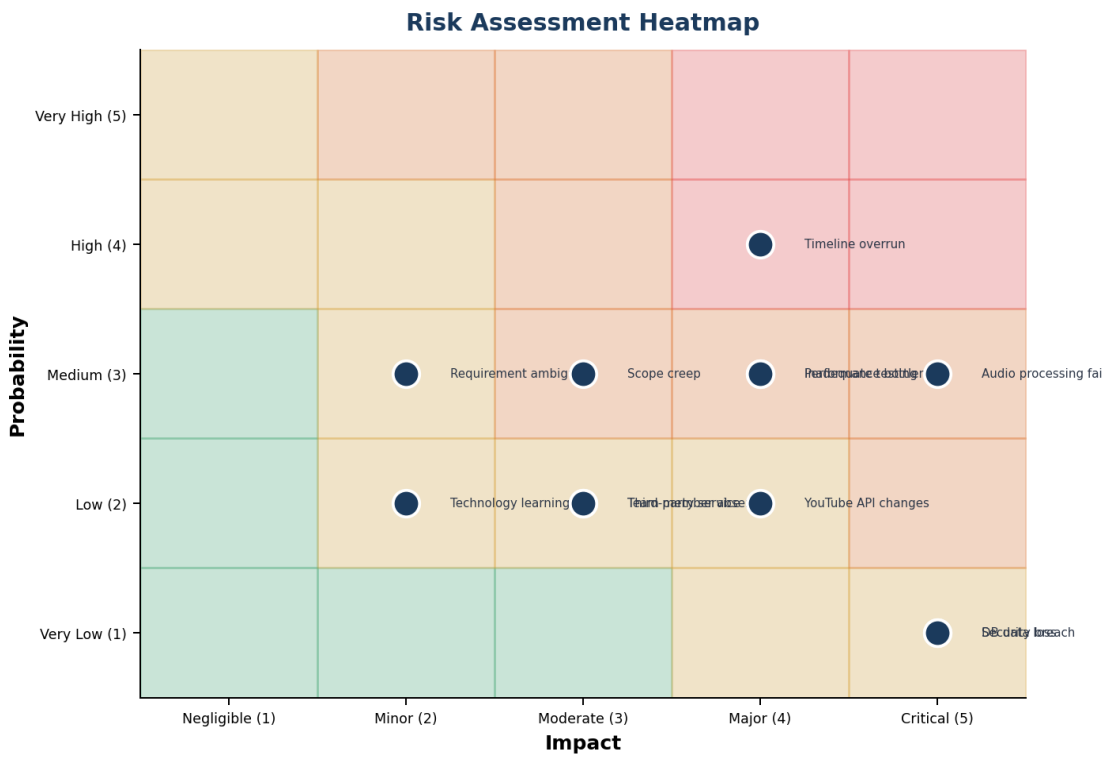

Figure 4.1: Risk assessment heatmap showing probability vs. impact distribution.

4.3 Mitigation Strategies

Risk	Category	Mitigation Strategy
R01	Audio processing	Implement retry logic (3 attempts), fallback FFmpeg versions, comprehensive error logging, and async job queue to prevent blocking
R06	Performance	Redis caching for hot data, CDN for audio delivery, DB indexing strategy, load testing with k6, and pagination for all list endpoints
R08	Timeline	MoSCoW prioritization, weekly progress tracking, buffer time in sprint planning, and early identification of blockers
R11	Testing	Automated CI pipeline (GitHub Actions), minimum 60% unit test coverage, integration tests for critical paths, E2E tests for key scenarios
R03	Security	OWASP guidelines, JWT with short expiry, bcrypt password hashing, parameterized queries, rate limiting, and HTTPS enforcement

Table 4.2: Mitigation strategies for high-priority risks.

5. Key Performance Indicators

5.1 Project Management KPIs

KPI	Description	Target	Tracking Method
Task Completion Rate	% of planned tasks completed per sprint	>= 95%	Sprint review
On-Time Delivery	% of milestones delivered on schedule	>= 90%	Milestone tracking
Budget Adherence	Actual vs. estimated hours per module	Within 10%	Weekly tracking
Sprint Velocity	Story points completed per sprint	Trending upward	Burndown chart
Defect Density	Bugs per module after sprint review	< 3 per sprint	Bug tracking

Table 5.1: Project management KPIs with targets.

5.2 Technical KPIs

KPI	Description	Target	Measurement
API Response Time	Average response time for standard endpoints	< 500ms (p95)	APM monitoring
Page Load Time	Time to interactive for any public page	< 2 seconds	Lighthouse CI
Search Performance	Full-text search query response time	< 1 second	Query logging
Test Coverage	Unit test code coverage percentage	>= 60%	Coverlet reports
Code Quality	SonarQube quality gate pass rate	A rating	SonarQube scan
Uptime	Platform availability during test period	>= 99%	Health checks

Table 5.2: Technical KPIs with measurable targets.

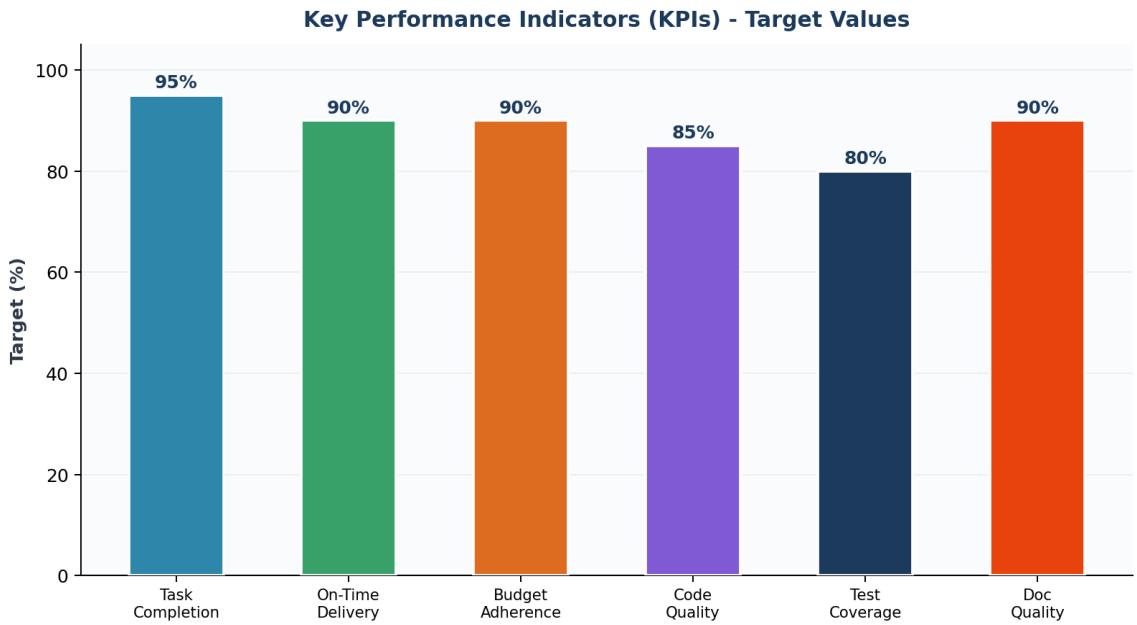

Figure 5.1: KPI target values visualization.

Version	Date	Author	Changes
1.0	2026-04-30	Project Team	Initial condensed release

Table 5.3: Document revision history.